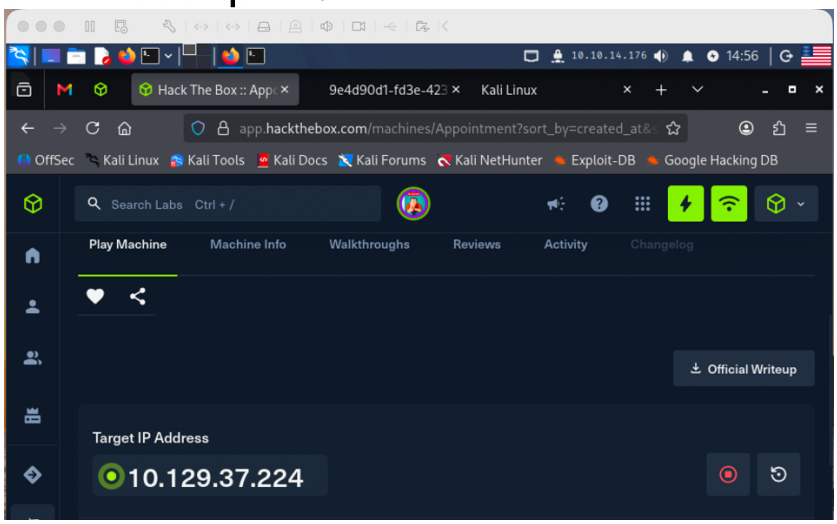
Hack The Box: Appointment Writeup (Tier 0 – Starting Point)

Дата проведения аудита: 11 октября 2026 г.
Платформа атаки: Kali Linux arm64 (Hypervisor: VMware Fusion Pro, Apple Silicon M4)
Целевой веб-хост: `10.129.37.224`
Результат: Успешный обход аутентификации (Authentication Bypass) через SQL-инъекцию (`SOLVED`), получен полный доступ к скрытому веб-контенту, эксфильтрован Root-флаг.



---
1. Введение и Теоретический анализ уязвимости
В ходе планового тестирования на проникновение исследовано веб-приложение на хосте `Appointment`. Данный аудит относится к сфере **Application Security (AppSec)** и направлен на выявление критических недостатков прикладного уровня (OWASP Top 10).

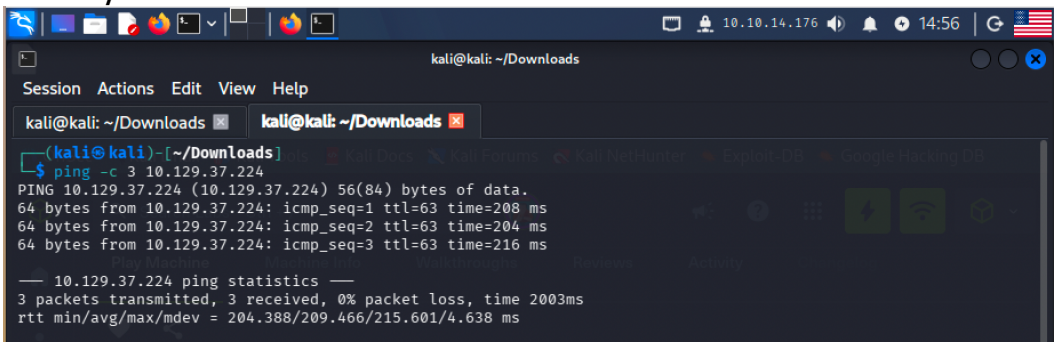
1.1. Специфика уязвимости SQL-Injection (SQLi)
Уязвимость SQL-инъекции является абсолютно универсальной уязвимостью прикладного уровня (Application Layer). Она возникает везде, где нарушено фундаментальное правило разработки: «Никогда не доверяй пользовательскому вводу». На объекте обнаружен критический недостаток отсутствия санитизации входящих данных

(Lack of Input Sanitization) в бэкенд-скрипте авторизации.

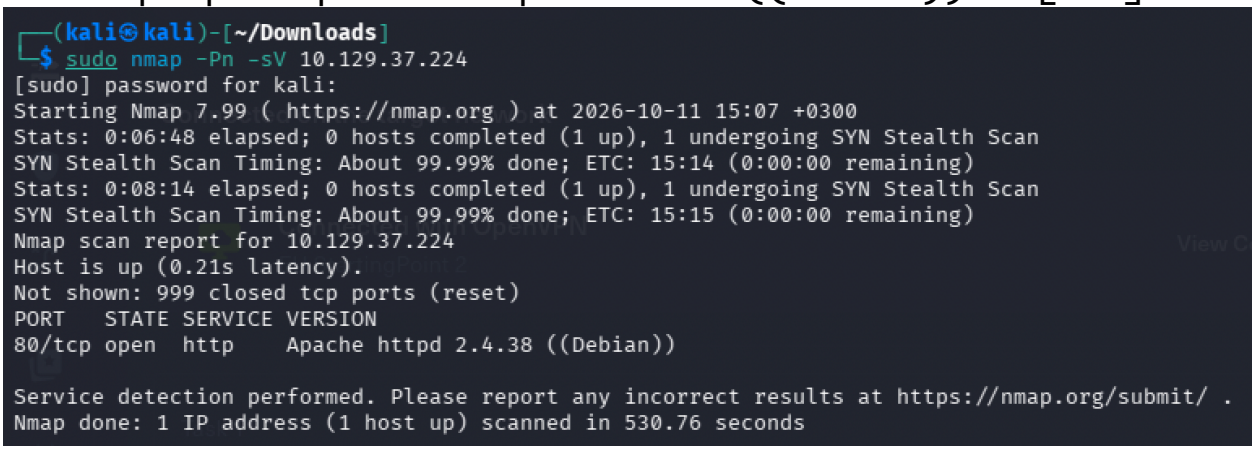
1.2. Физика дефекта безопасности
Исходный код PHP-обработчика принимал строковые переменные напрямую из POST-запроса и вставлял их в тело SQL-запроса в виде обычного текста (конкатенация строк) вместо безопасной передачи через параметры:
```php
$sql = "SELECT * FROM users WHERE username='$username' AND password='$password'";
```
Данная небезопасная конфигурация позволила атакующему внедрить сторонние SQL-команды, принудительно изменив логику выполнения инструкций бэкенд-СУБД (MySQL).

---
2. Этап разведки и перечисления (Reconnaissance & Fuzzing)

2.1. Тестирование сетевой доступности (ICMP Echo)
Перед началом сканирования портов проведена проверка стабильности зашифрованного сетевого канала с помощью утилиты `ping`:
```bash
ping -c 3 10.129.37.224
```
***Результат:** `3 packets transmitted, 3 received, 0% packet loss`. Среднее время задержки (RTT) составило ~209 мс, что подтвердило стабильность сетевого стека гипервизора.*

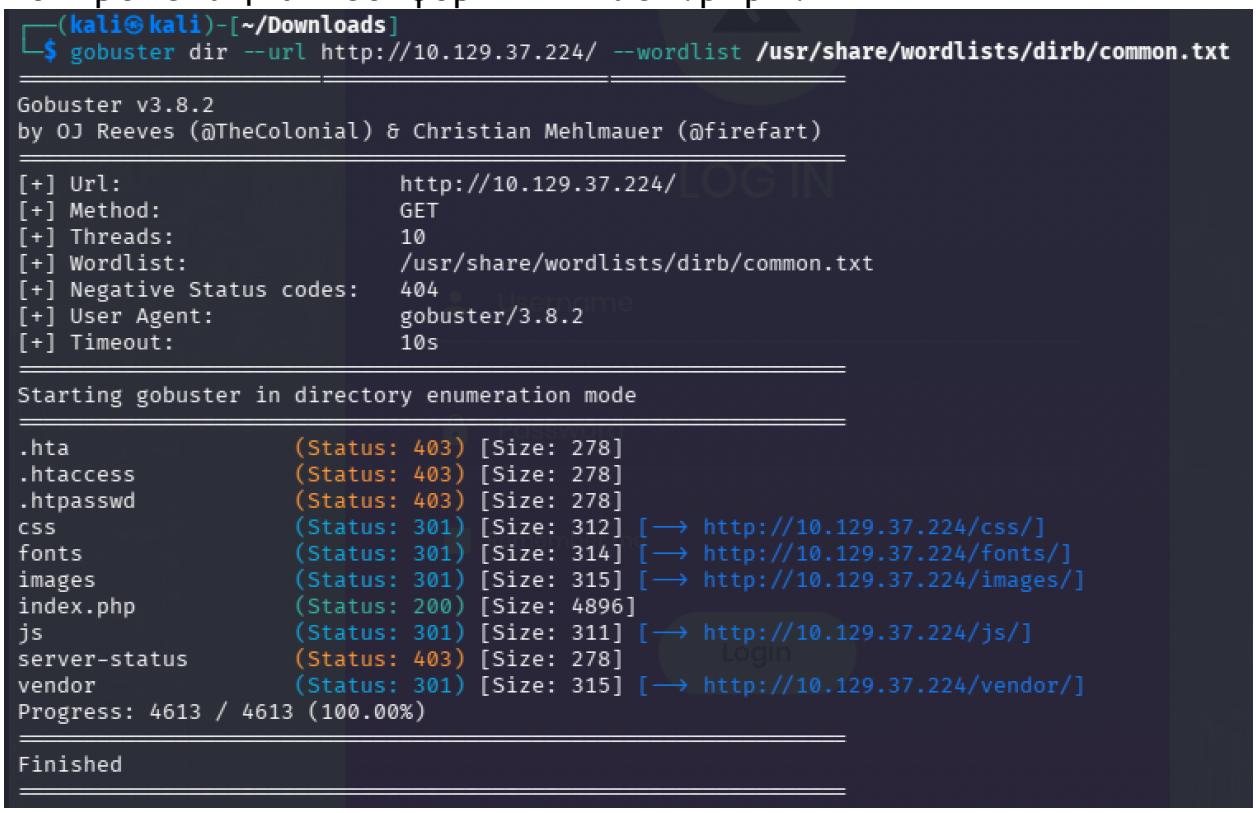


2.2. Сканирование портов (Nmap)
Для точечного обнаружения активных сетевых служб выполнено сканирование веб-порта с определением версий ПО (`-sV`) и отключением предварительного пингования (`-Pn`):
```bash
sudo nmap -Pn -sV 10.129.37.224
```
Лог сканирования Nmap:
```text
PORT   STATE SERVICE VERSION
80/tcp open  http    Apache httpd 2.4.38 ((Debian))
```
Сканирование официально подтвердило: порт **80/tcp** находится в открытом состоянии (`open`), на нем развернут веб-сервер **Apache httpd 2.4.38 ((Debian))** [1.8].



2.3. Автоматизированный перебор директорий (GoBuster)
Для выявления скрытых каталогов и потенциальных векторов атак был произведен автоматизированный перебор веб-структуры с помощью утилиты `gobuster` в режиме `dir` с использованием системных словарей Kali Linux из папки `/usr/share/wordlists`:
```bash
gobuster dir --url http://10.129.37 --wordlist /usr/share/wordlists/dirb/common.txt
```
Результат сканирования:

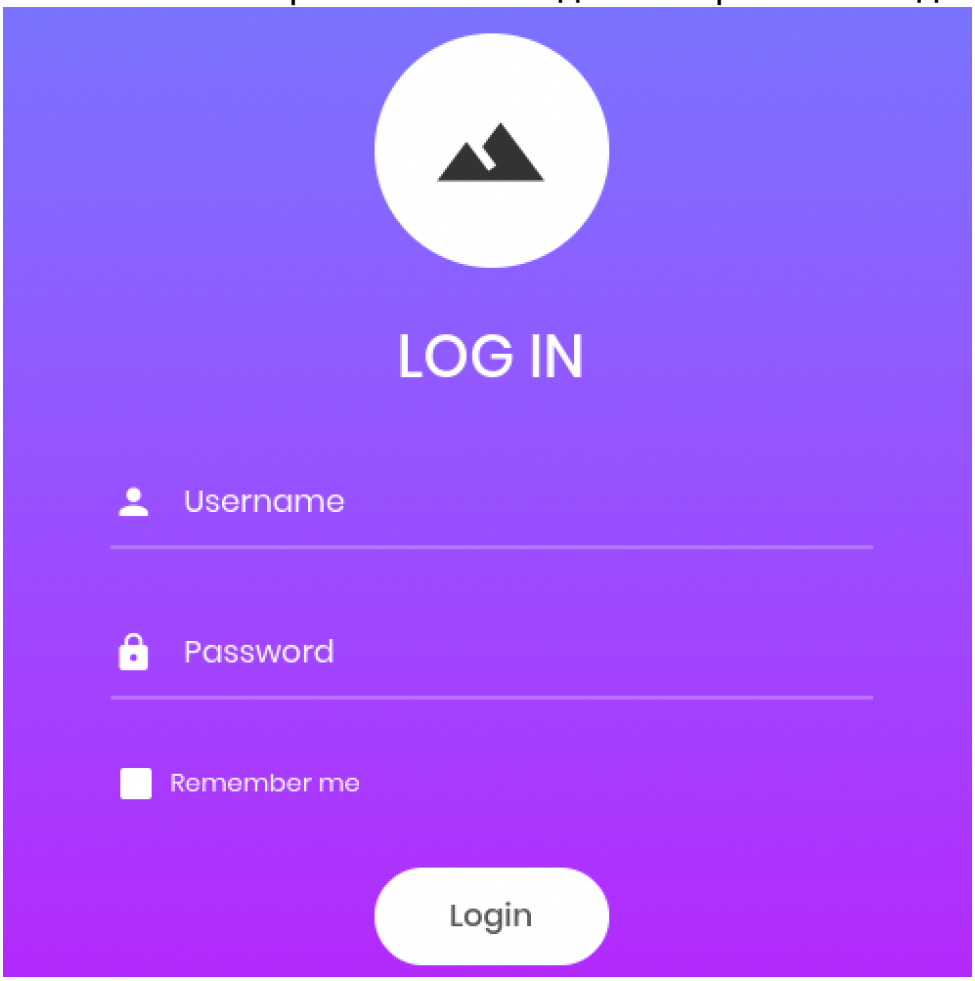
* `/index.php` (Status: 200, Size: 4896) – Главная веб-форма авторизации.
* `/css/`, `/js/`, `/vendor/` (Status: 301) – Стандартные каталоги стилей и библиотек.
Дополнительных скрытых панелей администрирования не обнаружено, основной вектор направлен на прямую компрометацию веб-формы `index.php`.



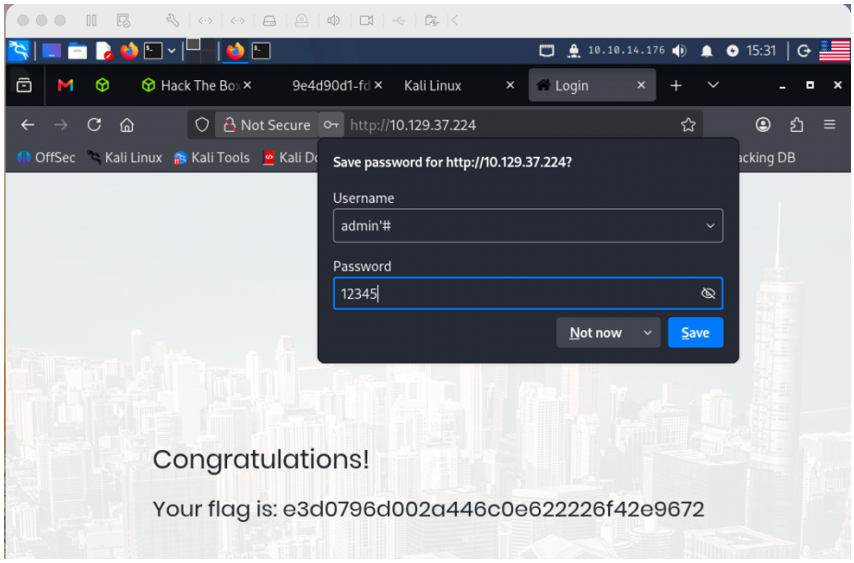
---
3. Этап эксплуатации (Exploitation)
3.1. Реализация вектора Authentication Bypass
Для обхода проверки пароля без наличия легитимных учетных данных в поле `Username` веб-формы `LOG IN` был внедрен специально сформированный пейлоад [source: XmXq7y, yhdYMO]:
```text
admin'#
```
В поле `Password` были введены произвольные символы (`12345`).

Механика пробива на стороне СУБД:

Одинарная кавычка принудительно закрыла строковый литерал, а символ решетки (**`#`**) заставил интерпретатор MySQL превратить всю оставшуюся часть SQL-запроса (включая проверку пароля) в обычный комментарий:
```sql
SELECT * FROM users WHERE username='admin'#' AND password='12345'
```
База данных полностью проигнорировала проверку пароля, вернула истину для пользователя `admin`, и веб-приложение предоставило неавторизованный административный доступ.



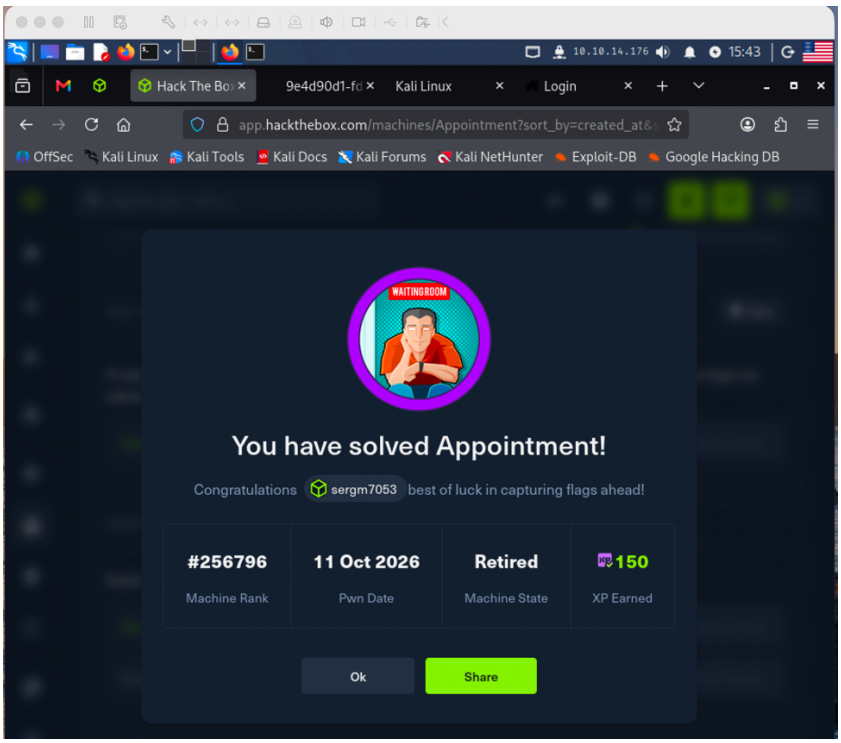
3.2. Извлечение Root-флага
После успешного байпаса страница перенаправила сессию на внутренний контур, где отобразилось приветственное слово **`Congratulations!`** и целевой трофей:
👉 **`e3d0796d002a446c0e622226f42e9672`**



---

4. Официальные ответы на контрольные вопросы (HTB Tasks)

* **Task 1 (Расшифровка SQL):** `Structured Query Language`
* **Task 2 (Индекс первого элемента массива):** `0`
* **Task 3 (Самый частый тип уязвимостей SQL):** `SQL injection`
* **Task 4 (Категория OWASP Top 10 2021):** `A03:2021-Injection`
* **Task 5 (Версия службы на порту 80 цели):** `Apache httpd 2.4.38 ((Debian))`
* **Task 6 (Стандартный порт протокола HTTPS):** `443`
* **Task 7 (Папка в веб-терминологии):** `directory`
* **Task 8 (HTTP код ошибки Not Found):** `404`
* **Task 9 (Флаг типа сканирования в gobuster):** `dir`
* **Task 10 (Символ комментария в MySQL):** `#`
* **Task 11 (Первое слово на возвращенной странице):** `Congratulations!`



---

5. Рекомендации по устранению уязвимостей (Remediation)

1. **Использование параметризованных запросов:** Полностью исключить прямую конкатенацию пользовательских строк в SQL-команды. Реализовать подготовленные выражения (**Prepared Statements**), где вводимые данные жестко обрабатываются базой данных как параметры, а не исполняемый код.
2. **Санитизация и валидация ввода:** Настроить строгую фильтрацию всех входящих данных на стороне бэкенда, блокируя или экранируя спецсимволы, такие как одинарные кавычки (`'`), двоеточия и знаки комментариев (`#`, `--`).
3. **Внедрение Web Application Firewall (WAF):** Развернуть на защитном контуре облачной инфраструктуры межсетевой экран уровня приложений (WAF), способный автоматически обнаруживать и блокировать сигнатуры SQLi-пейлоадов на подлете к веб-серверам.